

Ansible Core Interview Revision Guide

DevOps-focused • Core concepts • Production patterns • Best practices • Scenario-based interview questions

Target: Interview readiness, not deep Ansible specialization.

How to use this guide

- Revise the core concepts first.
- Use the production examples to understand where Ansible actually fits.
- Practice the scenario questions aloud before reading the model answer.
- Do not force Ansible into Kubernetes node autoscaling workflows when Kubernetes-native mechanisms are more appropriate.

Interview target: You should be able to explain Ansible architecture, inventory, playbooks, modules, variables, facts, handlers, templates, roles, Vault, Jenkins integration, Terraform comparison, and practical troubleshooting scenarios.

1. What Is Ansible?

Interview definition: Ansible is an agentless automation and configuration-management tool used to automate server configuration, application deployment, orchestration, and repetitive operational tasks.

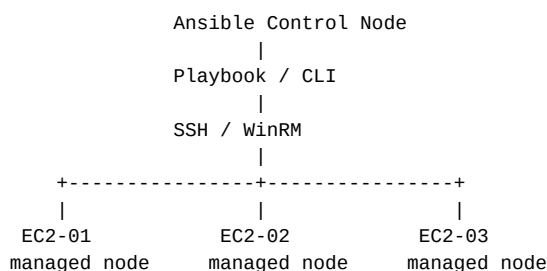
Keywords: agentless, YAML, SSH, modules, playbooks, idempotence, configuration management, automation.

Why companies use it

- Automates repetitive server and application operations.
- Uses human-readable YAML playbooks.
- Normally connects to Linux managed nodes over SSH without installing an Ansible agent.
- Supports repeatable, idempotent configuration.
- Can perform rolling and multi-tier deployments.

Production example: A company has 30 EC2 application servers. Instead of manually installing Nginx, creating users, copying configuration, and restarting services on every server, the team maintains an Ansible playbook and runs it against the target inventory.

2. Ansible Architecture



Component	What it means	Interview point
Control node	Machine from which Ansible is executed.	Can be a CI/CD runner or administration host.
Managed node	Target machine Ansible configures.	Usually no Ansible agent is required.
Inventory	List/grouping of managed hosts.	Determines where automation runs.
Playbook	YAML automation definition.	Describes desired configuration/tasks.
Module	Unit of work executed by Ansible.	Examples: service, package, template, copy.

3. Agentless Architecture

For typical Linux administration, Ansible uses existing remote access such as SSH rather than requiring a persistent Ansible agent on every server. This reduces software-management overhead on managed hosts.

Interview question: Why is Ansible called agentless?

Answer: Because managed Linux systems normally do not require an Ansible agent installed and running continuously. The control node connects remotely, commonly using SSH, and executes the required modules/tasks.

Do not overstate it: Agentless does not mean Ansible never needs any remote prerequisites. SSH access, runtime requirements for many modules, credentials, permissions, and network reachability still matter.

4. Inventory

Inventory defines the hosts Ansible can manage and lets you group hosts by function, environment, region, or other operational criteria. Ansible supports static inventories and dynamic inventory plugins.

```
[webservers]
web01
web02

[dbservers]
db01
db02

[production:children]
webservers
dbservers
```

Production pattern: Separate staging and production inventories or use controlled inventory sources so that a production playbook cannot accidentally target test systems.

Dynamic inventory: In cloud environments, inventory can be generated from cloud resources rather than manually maintaining a list of IP addresses. This is useful when instances are created or replaced dynamically.

Best practices

- Group by function such as web, app, and database.
- Separate production and non-production targets.
- Keep inventory and variables in version control.
- Use dynamic inventory for frequently changing cloud infrastructure.
- Use limiting patterns such as `-l appservers` when appropriate.

5. Ad-Hoc Commands

Ad-hoc commands are quick, one-off operations. They are useful for investigation or simple administrative actions, but repeatable production changes should generally be captured in playbooks.

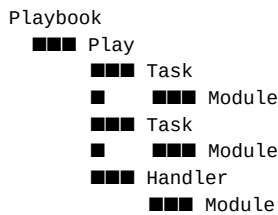
```
ansible all -i inventory.ini -m ping

ansible webservers -i inventory.ini -m service -a "name=nginx state=started"
```

Interview distinction: Ad-hoc command = quick one-time operation. Playbook = repeatable, reviewable automation.

6. Playbooks, Plays, Tasks and Modules

The hierarchy is one of the most important things to understand:



A playbook is an automation blueprint, a play is an ordered set of tasks mapped to hosts, a task is a reference to a module, and a module performs the actual unit of work.

```
- name: Configure web server
  hosts: webservers
  become: true

  tasks:
    - name: Install nginx
      ansible.builtin.package:
        name: nginx
        state: present

    - name: Start nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

Production example: A standard web-server role may install the required package, create directories/users, render configuration files, enable the service, and verify the service is running.

7. Idempotence

Idempotence means that when the managed host already matches the desired state, running the same automation again should not make unnecessary changes.

Example: `state: present` should not reinstall an already installed package every time the playbook runs.

Why it matters: Production automation may be executed repeatedly by CI/CD, scheduled jobs, or incident-response workflows. Idempotent automation makes repeated execution safer and more predictable.

Interview question: What happens if you run an Ansible playbook twice?

Answer: If the playbook is written idempotently and the target is already in the desired state, most tasks should report ok rather than changed. Poorly designed shell/command tasks can break this expectation, so modules and explicit change conditions are preferred.

8. Variables and Facts

Variables make playbooks reusable across environments and hosts. Facts are system information gathered from managed nodes, such as OS family, hostname, interfaces, and hardware details.

```
vars:
  app_port: 8080
  app_name: payments

tasks:
  - name: Show application port
    ansible.builtin.debug:
      msg: "Application port is {{ app_port }}"

  - name: Install package on Red Hat family
    ansible.builtin.package:
```

```
name: nginx
state: present
when: ansible_os_family == "RedHat"
```

Know these variable sources: vars, vars_files, group_vars, host_vars, role defaults/vars, registered variables, and extra vars. For interview purposes, understand that Ansible has variable precedence rather than trying to memorize the entire precedence table.

9. register, when, changed_when, failed_when

```
- name: Check application health
  ansible.builtin.uri:
    url: http://localhost:8080/health
    return_content: false
  register: health_result

- name: Stop deployment on unhealthy app
  ansible.builtin.fail:
    msg: "Application health check failed"
  when: health_result.status != 200
```

register: stores task output in a variable.

when: controls whether a task runs.

changed_when / failed_when: allow you to explicitly define how Ansible should classify command results when the module's default behavior is insufficient.

10. Handlers

```
tasks:
- name: Deploy nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart nginx

handlers:
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Handlers are special tasks triggered by notifications. They are commonly used for actions such as restarting or reloading a service only when its configuration changed.

Best practice: Do not restart a production service on every run if a configuration change did not occur. Use handlers for change-triggered service actions.

11. Templates and Jinja2

Templates allow configuration files to be generated dynamically from variables and logic. Ansible commonly uses Jinja2 syntax.

```
server {
    listen {{ app_port }};
    server_name {{ server_name }};
}
```

The same template can produce different staging and production configurations by changing variables rather than maintaining separate hard-coded files.

12. Roles

Roles provide a reusable structure for organizing Ansible automation. They separate tasks, handlers, templates, files, variables, defaults, and metadata into predictable directories.

```
roles/
├── nginx/
│   ├── tasks/
│   │   └── main.yml
│   ├── handlers/
│   │   └── main.yml
│   ├── templates/
│   ├── files/
│   ├── vars/
│   ├── defaults/
│   └── meta/
```

Production example: Instead of maintaining separate 500-line playbooks for different teams, an organization may maintain reusable roles such as `nginx`, `docker`, `node_exporter`, and `app_deploy`.

Interview answer: Roles make automation modular, reusable, easier to review, and easier to maintain as the Ansible codebase grows.

13. Ansible Vault and Secrets

Ansible Vault encrypts sensitive variables/files so secrets are not stored as plaintext. It is appropriate for protecting data at rest in an Ansible repository, but it is not a complete runtime secret-management solution.

```
ansible-vault create secrets.yml
ansible-vault edit secrets.yml
ansible-vault encrypt existing.yml
ansible-vault rekey secrets.yml
```

Production practice: Many organizations prefer a dedicated secret manager for application/runtime secrets, such as AWS Secrets Manager or HashiCorp Vault. If Ansible Vault is used, protect the Vault password and prevent secret values from appearing in logs.

Interview trap: Ansible Vault encrypts secrets at rest; once decrypted/used, you still need controls such as `no_log` and careful handling to prevent disclosure.

14. Ansible + Jenkins in a CI/CD Pipeline

```
Developer
|
GitHub
|
Jenkins
|
+-----+
| Build / Test / Quality Checks |
+-----+
|
Ansible playbook
|
EC2 application servers
|
Application configured/deployed
```

Jenkins is the orchestrator; Ansible performs configuration/deployment automation on the target hosts. A Jenkins stage can invoke an Ansible playbook after the CI checks pass.

```
stage('Deploy') {
    steps {
        sh 'ansible-playbook -i inventory/prod deploy.yml'
    }
}
```

Production controls: Use protected production environments, approval gates where required, immutable/versioned artifacts, least-privilege credentials, deployment logging, and a rollback strategy.

15. Ansible vs Terraform

Question	Terraform	Ansible
Primary focus	Provision/manage infrastructure as code	Configuration, deployment, orchestration
Typical target	AWS resources, networking, IAM, databases, servers	Servers software, configuration, application operations
Example	Create VPC + EC2	Install Docker + configure service
State model	Maintains infrastructure state	Primarily executes desired configuration/tasks
Typical relationship	Provision infrastructure	Configure/use provisioned hosts

Interview-ready explanation: Terraform is typically used to provision infrastructure such as VPCs, subnets, EC2, IAM, and EKS. Ansible is typically used to configure machines and automate software/application operations on systems that need configuration. They can be used together, but neither tool has to be present in every architecture.

16. Where Ansible Fits with EKS, Cluster Autoscaler and Karpenter

Traditional EC2 application:

Terraform → EC2 → Ansible → Application

EKS with managed nodes:

Terraform → EKS/Node Group → EC2 → EKS bootstrap → Kubernetes node

EKS with Cluster Autoscaler:

Kubernetes demand → Cluster Autoscaler → node capacity → EC2 → EKS node

EKS with Karpenter:

Pending pods → Karpenter → EC2 capacity → EKS node

Key point: You generally do not run Ansible as a per-node configuration step every time Karpenter or Cluster Autoscaler creates an EKS worker node. EKS node image/bootstrap and Kubernetes-native mechanisms are normally responsible for the node lifecycle. Ansible can still be used elsewhere in the platform for supporting EC2 hosts, legacy systems, configuration management, or other operational automation.

17. Production Best Practices

Practice	What to do	Why
Version control	Keep playbooks, roles and inventory definitions in Git.	Auditability, review, rollback.
Use roles	Break reusable automation into roles.	Maintainability and reuse.
Prefer modules	Use purpose-built modules over raw shell commands where possible.	Settle idempotence and predictable behavior.
Use FQCN	Prefer names such as <code>ansible.builtin.service</code> .	Clear module ownership and compatibility.
Separate environments	Use controlled staging/prod inventories or inventory sources.	Reduce accidental production changes.
Dynamic inventory	Use cloud inventory plugins where hosts change frequently.	Avoid stale host lists.
Least privilege	Use dedicated identities and minimum required permissions.	Reduce blast radius.
Protect secrets	Use Vault and/or an enterprise secret manager; never commit secrets.	Security.
no_log	Hide sensitive task output when necessary.	Avoid secret leakage in logs.
Staging first	Test changes in non-production before production.	Reduce deployment risk.
Rolling updates	Use batches/serial where appropriate.	Maintain service availability.
Validate	Use syntax checks, linting, tests and controlled check mode.	Catch errors early.
Idempotence	Make repeated runs safe.	Reliable automation.
Descriptive names	Name plays, tasks and handlers clearly.	Troubleshooting and readability.

Useful command: `ansible-playbook -i inventory/prod site.yml --check`

Check mode can help preview intended changes, but it is not a perfect substitute for testing in a real environment; modules may have check-mode limitations and external side effects should still be considered.

18. Production Scenario Walkthroughs

Scenario A — Configure 20 EC2 application servers

Requirement: Install Docker, create an application user, copy configuration, deploy the application, and start the service on 20 EC2 instances.

```
Terraform
  ↓
EC2 instances
  ↓
Dynamic/static Ansible inventory
  ↓
Ansible role
  ■■■ users
  ■■■ docker
  ■■■ configuration template
  ■■■ application deployment
  ■■■ service handler
```

Best approach: Provision infrastructure with Terraform, expose the hosts through an appropriate inventory source, then run a version-controlled Ansible playbook/role from a controlled execution environment such as Jenkins or Ansible Automation Platform.

Scenario B — Configuration changed, but service should restart only when needed

Use a template task with notify and a handler. If the rendered file is unchanged, the handler is not triggered.

Scenario C — One server has a different OS

Use facts and conditional logic or OS-specific variables/roles. Prefer platform-appropriate modules and keep OS-specific differences isolated rather than scattering many conditions throughout every task.

Scenario D — Production deployment must not hit every server at once

Use controlled batching/rolling execution, health checks, and failure handling. Update a small subset, verify health, then continue. This reduces blast radius during a production rollout.

Scenario E — Ansible playbook fails halfway through

Identify the failing host/task from the play recap and logs. Fix the underlying issue, then rerun the playbook. Idempotent tasks should allow the playbook to converge the environment without manually undoing every successful step. For risky operations, use blocks/rescue/always and explicit validation.

Scenario F — Karpenter launches a new EKS node

Do not assume an Ansible playbook must run against the node. The EKS node image/bootstrap and Karpenter/node configuration should make the node ready for Kubernetes. Ansible may be used elsewhere in the environment, but attaching an Ansible run to every autoscaled Kubernetes node is usually unnecessary and adds operational coupling.

19. Scenario-Based Interview Questions with Model Answers

1. You have 50 EC2 servers. How would you install Nginx on all of them?

I would group the servers in an Ansible inventory and create an idempotent playbook using the package/service modules. I would test it in staging first and then run it against the production inventory. If the servers are dynamically created in AWS, I would consider dynamic inventory rather than maintaining IP addresses manually.

2. What happens if you run the same playbook twice?

If the tasks are idempotent and the target is already in the desired state, the second run should normally report no unnecessary changes. I would avoid raw shell commands where a proper module can provide better state handling.

3. Why would you use a role?

A role provides a reusable structure for tasks, handlers, templates, files, variables and defaults. It makes a larger Ansible codebase modular and maintainable.

4. Your Nginx configuration changes. How do you restart Nginx only when the file changes?

I would use the template or copy task with notify and define a handler that restarts or reloads Nginx. The handler executes only when the notifying task reports a change.

5. How would you handle different values for dev, staging and production?

I would externalize environment-specific values into appropriate inventory/group variables, host variables, role defaults/vars, or variable files, while keeping the playbook logic reusable. Production inventory should be protected from accidental targeting.

6. Where would you store database passwords?

I would avoid plaintext secrets in Git. Depending on the organization's standard, I would use Ansible Vault or preferably an enterprise secret manager such as AWS Secrets Manager. I would also prevent secret values from appearing in logs.

7. What is the difference between copy and template?

Copy transfers a mostly static file. Template renders a Jinja2 template using variables and logic before placing the resulting file on the target host.

8. What is register used for?

Register captures the result of a task into a variable. I can then use that output for conditions, validation, debugging, or subsequent tasks.

9. What is the difference between when and changed_when?

when decides whether a task should run. changed_when controls whether Ansible considers a task changed, which is useful when wrapping commands or interpreting custom operations.

10. Why prefer modules over shell/command?

Purpose-built modules generally provide structured parameters, better idempotence, clearer behavior and more predictable change reporting. Shell/command is useful when there is no suitable module or for specific OS commands, but it should not be the default choice.

11. How does Jenkins use Ansible?

Jenkins can orchestrate the pipeline and invoke an ansible-playbook command after build, test and quality stages. Jenkins handles pipeline orchestration while Ansible performs configuration or deployment automation on target hosts.

12. Terraform created an EC2 instance. What can Ansible do next?

Ansible can configure the instance: install packages, create users, configure services, render configuration files, deploy application components and validate the resulting state. This is a common Terraform-plus-Ansible pattern for traditional

EC2 workloads.

13. Would you run Ansible every time Karpenter creates an EKS worker node?

Generally no. Karpenter and the EKS node bootstrap/image configuration are designed to make dynamically provisioned nodes ready for Kubernetes. Running Ansible as a mandatory per-node step would add latency and coupling. I would use Ansible where there is a real configuration-management requirement outside the normal EKS node lifecycle.

14. What is dynamic inventory?

Dynamic inventory obtains the current set of managed hosts from an external source, such as a cloud provider, rather than relying only on a manually maintained static host list. It is useful when infrastructure changes frequently.

15. How would you safely deploy to 100 production servers?

I would use version-controlled automation, staging validation, protected production credentials, controlled batches or rolling deployment, health checks, failure handling, observability and a rollback plan. I would avoid updating all servers simultaneously unless the architecture specifically requires it.

16. An Ansible task failed on one server. What would you do?

I would inspect the failing task, host, module output and connectivity/permission issues. After fixing the root cause, I would rerun the playbook because idempotent tasks should converge the host without requiring manual repetition of already successful steps.

17. How do you troubleshoot UNREACHABLE?

I would check DNS/IP resolution, network connectivity, security groups/firewalls, SSH port access, username, SSH key/credential configuration, bastion/proxy requirements and host availability. I would distinguish an SSH/connectivity problem from an application-level task failure.

18. How do you troubleshoot a task that says FAILED?

I would read the module error, confirm the target state and permissions, validate variables and inputs, and reproduce the smallest failing operation if necessary. I would avoid immediately adding `ignore_errors` because that can hide real deployment failures.

19. What is Ansible Vault's limitation?

Vault protects encrypted data at rest. Once the secret is decrypted and used, it can still leak through logs, debug output or application behavior. So Vault does not replace careful secret-handling practices.

20. How would you explain Ansible in one minute to an interviewer?

Ansible is an agentless automation and configuration-management tool. A control node uses inventories to select managed hosts and runs YAML playbooks made of tasks and modules. In production it can configure servers, deploy applications and orchestrate operational workflows. I would typically combine Terraform for infrastructure provisioning, Ansible for server configuration where needed, and Kubernetes-native tools such as Helm or Argo CD for Kubernetes workload delivery.

20. Rapid-Fire Interview Revision

Question	One-line answer
What is Ansible?	Agentless automation/configuration-management tool.
What is inventory?	The hosts/groups Ansible targets.
What is a playbook?	YAML automation blueprint.
What is a play?	Maps a set of tasks to selected hosts.
What is a task?	A single operation using a module.
What is a module?	Unit of work executed by Ansible.
What is idempotence?	Repeated runs converge without unnecessary changes.
What are facts?	System information gathered from managed hosts.
What is register?	Stores task output in a variable.
What is a handler?	A task triggered by notification, often after configuration changes.
What is a template?	A Jinja2 file rendered with variables/logic.
What is a role?	Reusable structure for organizing Ansible automation.
What is Vault?	Encryption mechanism for sensitive Ansible data at rest.
Ansible vs Terraform?	Terraform manages infrastructure; Ansible commonly configures/automates systems.
Ansible + Jenkins?	Jenkins orchestrates; Ansible performs the automation.
Ansible + Karpenter?	Usually not required for each dynamically created EKS node.

21. 30-Second Architecture Answer

Use this structure in interviews:

"Terraform provisions the AWS infrastructure.
For traditional EC2 workloads, Ansible can then configure the servers and deploy required software.

Jenkins can orchestrate the CI/CD workflow and invoke Ansible playbooks.

For EKS workloads, Kubernetes-native mechanisms such as Helm or Argo CD are typically used for application delivery, while managed node groups, Cluster Autoscaler or Karpenter handle worker-node lifecycle. I would not automatically put Ansible into the EKS node autoscaling path."

22. Final Revision Checklist

- I can define Ansible in 30 seconds.
- I can explain control node, managed node and agentless architecture.
- I can create/read a basic inventory.
- I know ad-hoc command vs playbook.
- I understand play → task → module.
- I can explain idempotence with a real example.

- I understand variables, facts, register and when.
- I can explain handlers and templates.
- I know why roles are used.
- I know how Vault protects secrets at rest.
- I can explain Jenkins + Ansible.
- I can explain Terraform + Ansible.
- I know where Ansible fits—and does not fit—in EKS.
- I can troubleshoot unreachable hosts and failed tasks.
- I can describe a safe production rollout.

23. Official Reference Material

This guide is interview-focused. For authoritative details, use the current Ansible documentation:

- Ansible introduction: docs.ansible.com/projects/ansible/latest/getting_started/introduction.html
- Playbooks: docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks.html
- Inventory: docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html
- Ansible Vault: docs.ansible.com/projects/ansible/latest/vault_guide/vault.html
- Tips and best practices: docs.ansible.com/projects/ansible/latest/tips_tricks/index.html

Recommended next step: After revising this PDF, do a mock Ansible interview with one question at a time. Focus on explaining the concepts in your own words rather than memorizing the model answers.